EXPERIMENT-68(HDOC)

C Programming

Program: First n Fibonacci Numbers Using Recursion, with a Count of Recursive Calls

Step 1: Understand the Problem Statement

- Input: A positive integer n (the number of terms), from 1 to 40. Zero, a negative number, a number above 40 or a non-numeric value is rejected with an error message.
- Output: The first n Fibonacci numbers, displayed as `Fibonacci series: 0 1 1 2 ...`, followed by `Total recursive calls = count`.
- The Fibonacci series starts with 0 and 1, and every later term is the sum of the two terms before it. The k-th term is found with the recursive function `fib(int k)`, and the series is generated by calling it for k = 0, 1, ..., n-1.
- Base case: when k <= 1, return k (fib(0) = 0 and fib(1) = 1). Recursive case: otherwise return `fib(k - 1) + fib(k - 2)`. The argument decreases at every call, so the recursion always ends.
- To count the calls, a global counter `calls` is increased by 1 at the start of every call to `fib()`, so it also counts the calls that main() makes. For a single term, the number of calls is C(0) = C(1) = 1 and C(k) = 1 + C(k-1) + C(k-2), which grows very fast. The limit of 40 keeps the run time short and the values within the range of long long.

Rule(s) used: fib(k) = k when k <= 1; fib(k) = fib(k-1) + fib(k-2) when k >= 2. Example: fib(4) → fib(3) + fib(2) → ... = 3, and this one call makes 9 calls in total.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`Number of terms: 1`	Fibonacci series: 0 Total recursive calls = 1	Fibonacci series: 0 Total recursive calls = 1	Pass
2	`Number of terms: 5`	Fibonacci series: 0 1 1 2 3 Total recursive calls = 19	Fibonacci series: 0 1 1 2 3 Total recursive calls = 19	Pass
3	`Number of terms: 8`	Fibonacci series: 0 1 1 2 3 5 8 13 Total recursive calls = 100	Fibonacci series: 0 1 1 2 3 5 8 13 Total recursive calls = 100	Pass
4	`Number of terms: 10`	Fibonacci series: 0 1 1 2 3 5 8 13 21 34 Total recursive calls = 276	Fibonacci series: 0 1 1 2 3 5 8 13 21 34 Total recursive calls = 276	Pass
5	`Number of terms: 15`	Fibonacci series: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 Total recursive calls = 3177	Fibonacci series: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 Total recursive calls = 3177	Pass
6	`Number of terms: 0`	Invalid input. Enter an integer from 1 to 40.	Invalid input. Enter an integer from 1 to 40.	Pass
7	`Number of terms: -3`	Invalid input. Enter an integer from 1 to 40.	Invalid input. Enter an integer from 1 to 40.	Pass
8	`Number of terms: abc`	Invalid input. Enter an integer from 1 to 40.	Invalid input. Enter an integer from 1 to 40.	Pass

Step 3: Algorithm

1. Start the program and set the global counter calls = 0.
2. Read the number of terms n; if it is not an integer from 1 to 40, display an error message and stop.
3. Display the heading Fibonacci series:.
4. For i = 0 to n-1, call fib(i) and display the value returned.
5. In fib(): increase calls by 1.
6. If k <= 1, return k (base case).
7. Otherwise return fib(k - 1) + fib(k - 2) (recursive case).
8. After the loop, display Total recursive calls = calls.
9. Stop the program.

Evaluation against Step 2 test cases: for n = 1 only fib(0) is called, giving the series 0 with 1 call. For n = 5 the terms are fib(0) to fib(4) = 0 1 1 2 3, and the calls are 1 + 1 + 3 + 5 + 9 = 19, because fib(2), fib(3) and fib(4) need 3, 5 and 9 calls respectively. For n = 8 the calls are 1 + 1 + 3 + 5 + 9 + 15 + 25 + 41 = 100, and for n = 10 adding 67 and 109 gives 276. For n = 15 the series ends at 377 and the total is 3177, which shows how quickly the count grows. An input of 0, a negative number and a non-numeric entry are rejected as invalid input — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

```c
/* HDOC Day-69: Generate the first n Fibonacci numbers using RECURSION
   and count the number of recursive calls made.
   Fibonacci rule: fib(0) = 0, fib(1) = 1, fib(k) = fib(k-1) + fib(k-2)
   Base case     : k <= 1 -> return k
   Recursive case: k >= 2 -> return fib(k-1) + fib(k-2)
   A global counter is increased at the start of every call to fib(),
   so after the series is generated it holds the total number of calls.
*/
#include <stdio.h>

/* Global counter for the number of calls made to fib() */
long long calls = 0;

/* Function prototype */
long long fib(int k);

int main() {
    int n, i;

    printf("Enter number of terms: ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 40) {
        printf("Invalid input. Enter an integer from 1 to 40.\n");
        return 1;
    }

    printf("Fibonacci series:");
    for (i = 0; i < n; i++)
        printf(" %lld", fib(i));      /* i-th term, found recursively */
    printf("\n");

    printf("Total recursive calls = %lld\n", calls);
    return 0;
}

/* Returns the k-th Fibonacci number recursively and counts every call */
long long fib(int k) {
    calls++;                          /* one more call made */
    if (k <= 1)                       /* base case: fib(0) = 0, fib(1) = 1 */
        return k;
    return fib(k - 1) + fib(k - 2);   /* recursive case */
}
```

Step 5: Compile, Execute and Evaluate

The program was compiled using the cc compiler and executed from the Linux terminal using the following commands:

`cc hdoc69.c` (compile)
`./a.out` (execute)

All 8 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

